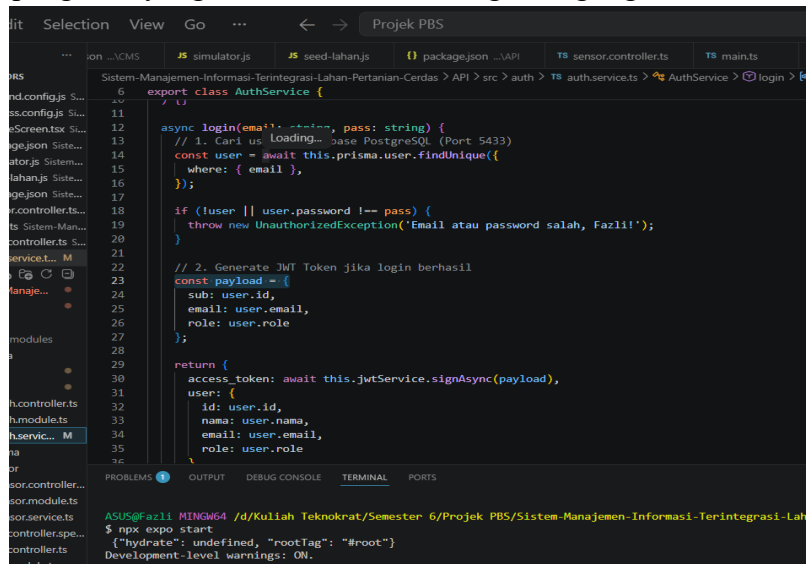


DOKUMENTASI SESI 2

Nama : M Fazli Anan

NPM 23312072

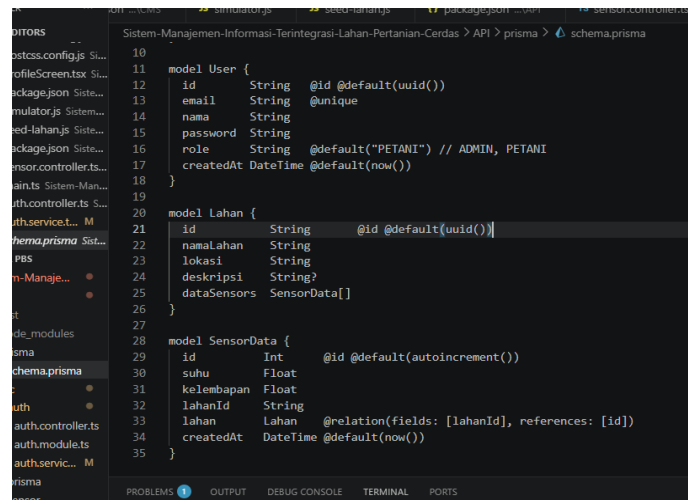
1. Mekanisme pembatasan hak akses dan proteksi resource data telemetry dikendalikan secara ketat melalui implementasi JwtAuthGuard pada level controller. Setiap request yang mengarah pada urusan manipulasi data lahan dan penarikan log sensor kini wajib melampirkan *Bearer Token* yang sah pada HTTP Header. Modul keamanan NestJS akan melakukan intersepsi dan memvalidasi tanda tangan digital JWT tersebut. Apabila token tidak disertakan atau telah kedaluwarsa, server akan menolak akses secara otomatis dengan memancarkan respon HTTP 401 *Unauthorized*. Proteksi berlapis ini menjamin bahwa hak kontrol atas cluster sistem smart farming TerraVision hanya dapat dieksekusi oleh akun pengelola yang sah setelah melewati gerbang login.



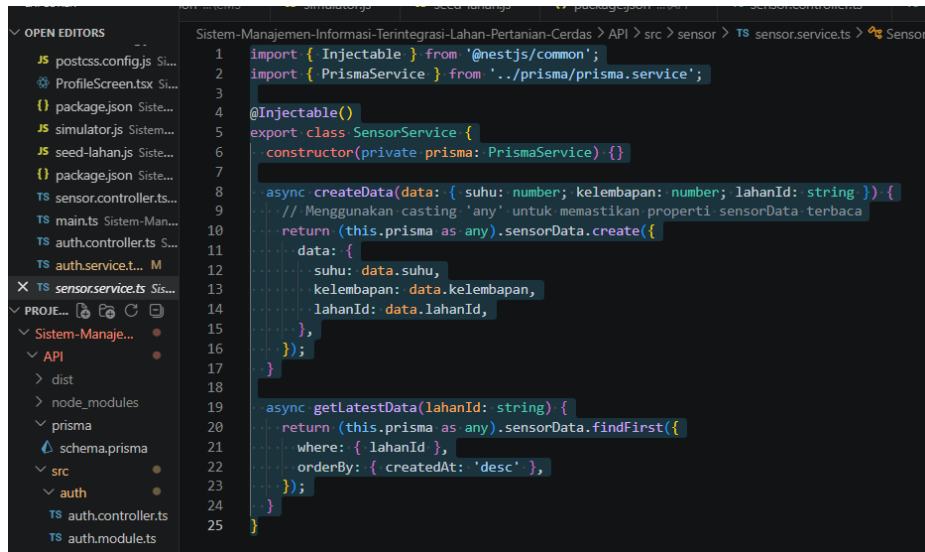
```
6 export class AuthService {
7
8   async login(email: string, pass: string) {
9     // 1. Cari user di database PostgreSQL (Port 5433)
10    const user = await this.prisma.user.findUnique({
11      where: { email },
12    });
13
14    if (!user || user.password !== pass) {
15      throw new UnauthorizedException('Email atau password salah, Fazli!');
16    }
17
18    // 2. Generate JWT Token jika login berhasil
19    const payload = {
20      sub: user.id,
21      email: user.email,
22      role: user.role
23    };
24
25    return {
26      access_token: await this.jwtService.signAsync(payload),
27      user: {
28        id: user.id,
29        nama: user.nama,
30        email: user.email,
31        role: user.role
32      }
33    };
34  }
35}
```

```
ASUS@fazli NING64 /d/Kuliah Teknokrat/Semester 6/Projek PBS/Sistem-Manajemen-Informasi-Terintegrasi-Lahan-Pertanian-Cerdas> API > src > auth > TS authService.ts > AuthService > login >
$ npm expo start
{"hydrate": undefined, "rootTag": "#root"}
Development-level warnings: ON.
```

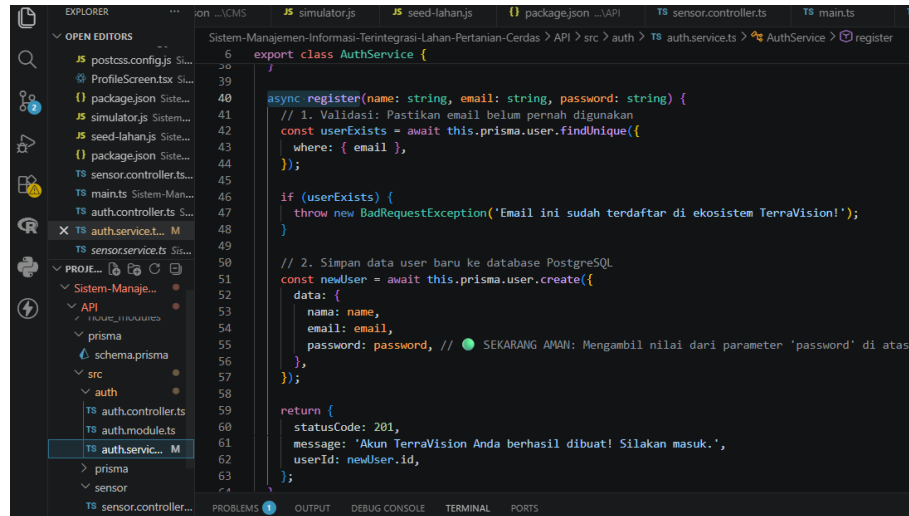
- Arsitektur relasional database diperluas pada skema Prisma ORM untuk mendukung pemetaan multi-blok lahan pertanian secara dinamis. Untuk memfasilitasi kebutuhan pemetaan area perkebunan, struktur tabel Lahan dikonfigurasi memiliki relasi *one-to-many* terhadap entitas SensorData. Setiap blok lahan (seperti Blok A untuk Kopi Robusta atau Blok B untuk Tomat) dapat mengasuh beberapa titik node sensor IoT sekaligus. Modifikasi skema ini disinkronisasikan langsung ke database PostgreSQL melalui perintah migrasi database Prisma, menghasilkan struktur relasi yang konsisten dan siap mendukung operasi penarikan data wilayah pertanian



- Penyediaan pipa data riwayat telemetri (*historical data stream*) diakomodasi melalui pengembangan endpoint temporal kueri. Guna menyuplai kebutuhan pengolahan visualisasi tren berkala, dikembangkan endpoint baru yaitu GET /sensor/history/:lahanId. Service ini bertugas mengalkulasi dan menarik kumpulan data agregat parameter log sensor (rata-rata fluktuasi kelembapan tanah, tingkat keasaman pH, dan suhu udara mikro) berdasarkan rentang waktu tertentu dari database pusat, sehingga riwayat kondisi vitalitas lahan dapat dianalisis secara kronologis.

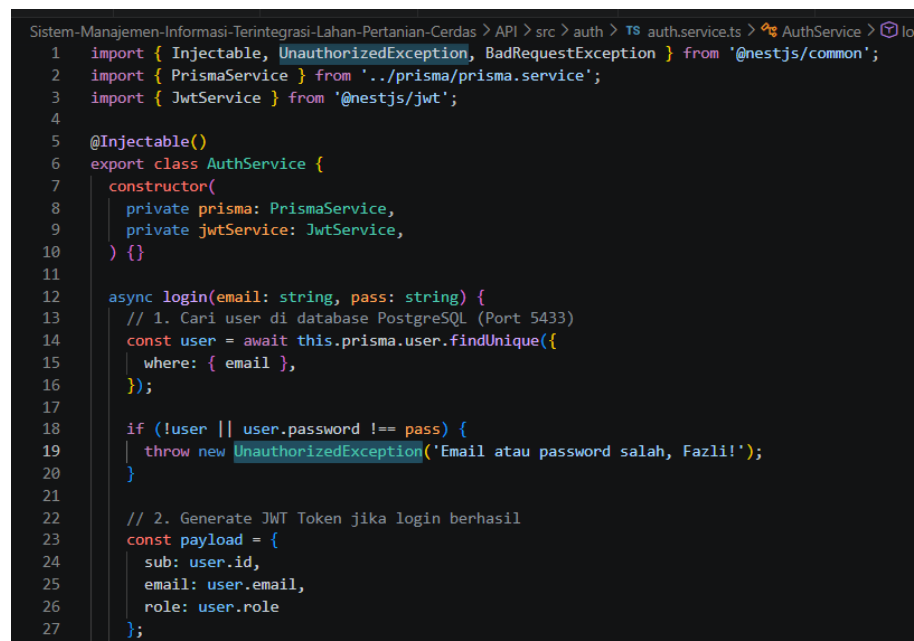


- Operasi mutasi data zona tanam diimplementasikan penuh melalui penyusunan logika bisnis pembuatan entitas lahan baru via HTTP POST. Fungsi `createNewZone` di dalam service mengekstraksi payload JSON yang dikirimkan oleh klien (berisi nama blok, koordinat, dan jenis komoditas) dan memprosesnya menggunakan metode `create` bawaan Prisma. Endpoint ini mengembalikan status HTTP 201 *Created* jika parameter valid, yang memicu sistem untuk memperbarui daftar data spasial lahan secara instan pada repositori basis data.



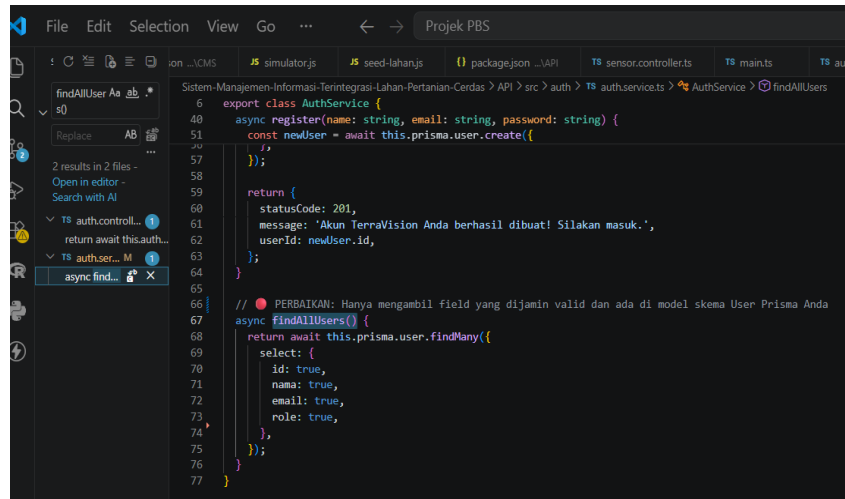
```
6 export class AuthService {
7
8   async register(name: string, email: string, password: string) {
9     // 1. Validasi: Pastikan email belum pernah digunakan
10    const userExists = await this.prisma.user.findUnique({
11      where: { email },
12    });
13
14    if (userExists) {
15      throw new BadRequestException('Email ini sudah terdaftar di ekosistem TerraVision!');
16    }
17
18    // 2. Simpan data user baru ke database PostgreSQL
19    const newUser = await this.prisma.user.create({
20      data: {
21        name: name,
22        email: email,
23        password: password, // SEKARANG AMAN: Mengambil nilai dari parameter 'password' di atas
24      },
25    });
26
27    return {
28      statusCode: 201,
29      message: 'Akun TerraVision Anda berhasil dibuat! Silakan masuk.',
30      userId: newUser.id,
31    };
32  }
33 }
```

- Penanganan anomali kegagalan database (*database exception handling*) diperkuat dengan menyematkan blok Try-Catch terisolasi pada pipa kueri. Guna mengantisipasi kondisi server database PostgreSQL *offline* atau terputus secara mendadak saat proses transaksi data sensor berlangsung, setiap fungsi vital dibungkus dengan interceptor *error handling*. Jika deteksi eror terjadi, sistem tidak akan mengalami mati total (*crash*), melainkan memancarkan respon kesalahan terkontrol lewat `HttpException` bawaan NestJS dengan status HTTP 500 *Internal Server Error*.



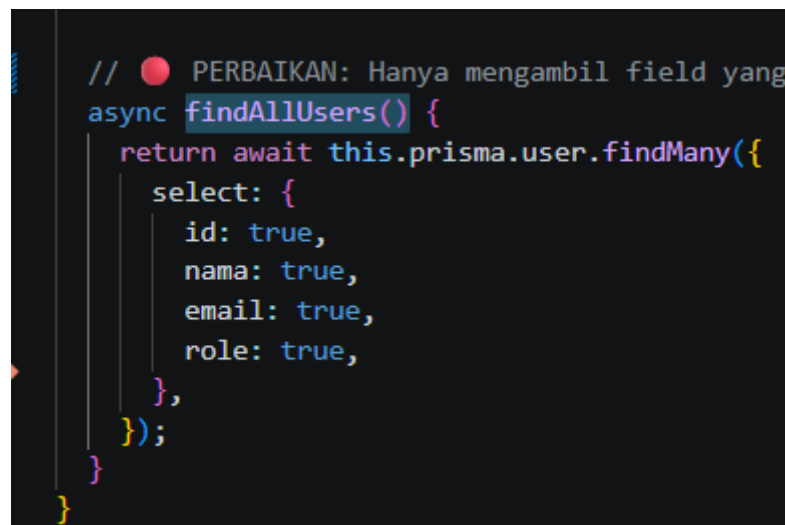
```
1 import { Injectable, UnauthorizedException, BadRequestException } from '@nestjs/common';
2 import { PrismaService } from '../prisma/prisma.service';
3 import { JwtService } from '@nestjs/jwt';
4
5 @Injectable()
6 export class AuthService {
7   constructor(
8     private prisma: PrismaService,
9     private jwtService: JwtService,
10   ) {}
11
12   async login(email: string, pass: string) {
13     // 1. Cari user di database PostgreSQL (Port 5433)
14     const user = await this.prisma.user.findUnique({
15       where: { email },
16     });
17
18     if (!user || user.password !== pass) {
19       throw new UnauthorizedException('Email atau password salah, Fazli!');
20     }
21
22     // 2. Generate JWT Token jika login berhasil
23     const payload = {
24       sub: user.id,
25       email: user.email,
26       role: user.role
27     };
28   }
```

6. Mekanisme penghapusan akun dan pembatalan registrasi pengguna dikelola secara aman via route DELETE /users/:id. Guna mengakomodasi modul manajemen otoritas akun pada CMS pengelola, dikembangkan endpoint yang memanfaatkan parameter pengidentifikasi unik berbasis *UUID*. NestJS akan memverifikasi keberadaan data pengguna sebelum mengeksekusi metode *delete* pada tabel untuk memastikan pembersihan record usang atau operator lahan yang sudah nonaktif berjalan lancar tanpa merusak dependensi tabel lain.



```
6 export class AuthService {
40   async register(name: string, email: string, password: string) {
51     const newUser = await this.prisma.user.create({
57     });
58   }
59   return {
60     statusCode: 201,
61     message: 'Akun TerraVision Anda berhasil dibuat! Silakan masuk.',
62     userId: newUser.id,
63   };
64 }
65
66 // ● PERBAIKAN: Hanya mengambil field yang dijamin valid dan ada di model skema User Prisma Anda
67 async findAllUsers() {
68   return await this.prisma.user.findMany({
69     select: {
70       id: true,
71       nama: true,
72       email: true,
73       role: true,
74     },
75   });
76 }
77 }
```

7. Proses pembaruan status verifikasi operator (*user state alteration*) diakomodasi melalui rute modifikasi parsial HTTP PATCH. Endpoint PATCH /users/:id/verify disusun untuk mendukung fungsionalitas pengubahan hak status keamanan akun secara instan dari status *Unverified* menjadi *Verified*. Berkas pengontrol menerima payload berisi status baru, lalu menginstruksikan Prisma ORM untuk mengompilasi perintah SQL *Update* parsial, mengubah parameter kolom keamanan di tabel database tanpa memodifikasi informasi profil dasar pengguna lainnya.



```
// ● PERBAIKAN: Hanya mengambil field yang
async findAllUsers() {
  return await this.prisma.user.findMany({
    select: {
      id: true,
      nama: true,
      email: true,
      role: true,
    },
  });
}
```

8. Penyusunan kerangka standarisasi objek data masukan dikontrol ketat melalui penerapan Data Transfer Object (DTO) dan ValidationPipe. Untuk menjaga integritas data sebelum menyentuh lapisan PostgreSQL, dibuat skema validator kelas seperti CreateUserDto. Dengan menyematkan decorator seperti `@IsEmail()` dan `@IsString()`, setiap data ilegal (seperti format surel tidak valid) yang dikirim dalam HTTP request body akan langsung dicegat di pintu gerbang controller dan dikembalikan dalam bentuk respon HTTP 400 *Bad Request* yang sistematis.

```
6 export class AuthService {
7   constructor(
8     private prisma: PrismaService,
9     private jwtService: JwtService,
10  ) {}
11
12  async login(email: string, pass: string) {
13    // 1. Cari user di database PostgreSQL (Port 5433)
14    const user = await this.prisma.user.findUnique({
15      where: { email },
16    });
17
18    if (!user || user.password !== pass) {
19      throw new UnauthorizedException('Email atau password salah, Fazli!');
20    }
21
22    // 2. Generate JWT Token jika login berhasil
23    const payload = {
24      sub: user.id,
25      email: user.email,
26      role: user.role
27    };
28  }
```

9. Konfigurasi kebijakan Cross-Origin Resource Sharing (CORS) diaktifkan secara global pada berkas inisialisasi server bootstrap `main.ts`. Mengingat arsitektur aplikasi TerraVision dikembangkan pada lingkungan terpisah—di mana server backend NestJS berjalan pada port terisolasi (port 3000)—kebijakan pembatasan asal request domain wajib dikonfigurasi secara manual. Pengaktifan metode `enableCors()` pada sistem backend krusial dilakukan agar aplikasi klien di luar server dapat melakukan konsumsi API lintas asal tanpa terblokir sistem keamanan internal peramban web.

```
File Edit Selection View Go ... < > Projek PBS
JS simulator.js JS seed-lahan.js {} package.json ...API TS sensor.co
enableCor Aa ab *
s
Replace AB
1 result in 1 file -
Open in editor -
Search with AI
TS main.ts Siste... 1
app.enableCors();
1 import { NestFactory } from '@nestjs/core';
2 import { AppModule } from './app.module';
3
4 async function bootstrap() {
5   const app = await NestFactory.create(AppModule);
6
7   app.enableCors();
8   // -----
9
10  await app.listen(process.env.PORT ?? 3000, '0.0.0.0');
11
12  console.log(`Application is running on: ${await app.getUrl()}`);
13 }
14 bootstrap();
```

10. Penggabungan modul-modul bisnis baru disinkronisasikan secara hierarkis ke dalam berkas konseptual `app.module.ts`. Sebagai orkestrator utama arsitektur *module-driven* NestJS, root module diperbarui untuk mendaftarkan modul perluasan Sesi 2 yaitu `LahanModule`. Dengan penggabungan terpusat ini, ketergantungan antar-modul (*dependency injection*) seperti pemakaian koneksi database bersama (`PrismaModule`) dan validasi keamanan JWT dapat dialokasikan dengan lancar ke seluruh layanan endpoint pintar, merampungkan seluruh kluster repositori backend untuk rilis fase kedua.



```
1 import { Module } from '@nestjs/common';
2 import { AppController } from './app.controller';
3 import { AppService } from './app.service';
4 import { PrismaModule } from './prisma/prisma.module';
5 import { SensorModule } from './sensor/sensor.module';
6 // 1. TAMBAHKAN IMPORT INI
7 import { AuthModule } from './auth/auth.module';
8
9 @Module({
10   // 2. MASUKKAN AuthModule KE DALAM ARRAY IMPORTS
11   imports: [PrismaModule, SensorModule, AuthModule],
12   controllers: [AppController],
13   providers: [AppService],
14 })
15 export class AppModule {}
```